

CSC 413 Project 2 Documentation

Summer 2026

Manisha Chand

924844476

Section 01

<https://github.com/csc413-SFSU-SU2025/interpreter-build4me2>

Table of Contents

1	Introduction.....	3
1.1	Project Overview.....	3
1.2	Technical Overview.....	3
1.3	Summary of Work Completed.....	3
2	Development Environment.....	3
3	Build or Import Instructions.....	3
4	Run Instructions.....	3
5	Assumptions.....	3
6	Implementation Discussion.....	3
6.1	UML Class Diagram.....	3
7	Project Reflection.....	3
8	Project Conclusion and Results.....	3

1 Introduction

1.1 Project Overview

This project is an interpreter for a made-up programming language called “language X.” This project reads bytecode and runs it. The two given programs compute a Fibonacci number and the factorial of a number written recursively; the interpreter correctly handles function calling itself recursively. Also, a few small, simple prompts in the project that build an array in dynamic memory, fill it with user-provided values, and then either print them or add them up. So basically, the program takes low-level instructions, keeps track of all the values and function calls those instructions produce, and produces the same result as the high-level program does.

1.2 Technical Overview

This project is a stack-based virtual machine that executes bytecode for a small language called X. It does not run .x source files directly; it runs the compiled bytecode files, where each line is one instruction such as LIT, BOP, CALL, or HALT. The program runs as a pipeline. Interpreter takes the .cod file name as a command-line argument and initializes the CodeTable, which maps each bytecode token to the class that implements it. The ByteCodeLoader then opens the file and, for each line, splits it into tokens, uses the CodeTable to find the right class, creates that ByteCode object by reflection, and passes the remaining tokens to the object's init method so it can store its own arguments. Every instruction is stored in order in a Program object, which holds them in an ArrayList<ByteCode>.

Before anything runs, Program.resolveAddress makes a pass over the list to handle jumps. The bytecode refers to jump targets by label name, so this step records the index of every LABEL and gives each GOTO, CALL, and FALSEBRANCH the actual instruction number it should jump to.

Execution happens in the VirtualMachine, holds all of the runtime state: the RunTimeStack, the Heap, a return-address stack, the program counter, and the running/verbose flags. Its executeProgram method is a simple fetch-execute loop, get the instruction at the program counter, call its execute, then advance the counter, and it keeps working so the loop never needs to know which specific instruction it is running.

The two memory structures are kept separate on purpose. The RunTimeStack holds function values using one Array of frame pointers, where each frame pointer marks where the current function's values begin. The Heap is independent dynamic memory used for array-like blocks that need to outlive a function call. The design rule throughout is that the bytecodes never touch; they request every operation through the Virtual Machine, which is also why VERBOSE mode is handled by the machine rather than by the instructions themselves.

1.3 Summary of Work Completed

1. All 19 bytecodes are implemented.
2. The codeTable, ByteCodeLoader, Program, RuntimeStack, heap, and VM classes are completed along with the required exception classes.
3. Address resolution for GOTO, CALL, and FALSEBRANCH works; labels are turned into instructions before the program runs.
4. The runtime stack protects all frame boundaries, and the heap does full safety checking.
5. Verbose mode prints each executed bytecode and a frame-by-frame view of the stack, and the output format matches the expectations.

2 Development Environment

Languages / JDK: Java 24, Open JDK

IDE: IntelliJ IDEA

OS: Linux

Version Control: Git/Github

3 Build or Import Instructions

Importing to IntelliJ IDEA

1. Clone the repository, open it in IntelliJ, and select the root of the repository: a folder that contains the interpreter folder.
2. Let IntelliJ make the repository root the sources root so the interpreter.* packages resolve correctly.
3. Set the project SDK to Java 16+(I used Java 24)

4 Run Instructions

The program takes one CLI argument: a .cod bytecode file to run.

With CLI, after building,

```
java -cp build interpreter.Interpreter factorial.x.cod
```

(May replace factorial.x.cod with any of the provided programs)

From IntelliJ IDEA

1. Create Eun config with Main class = interpreter.Interpreter
2. Set Program args to the filename, e.g., factorial.x.cod.
3. Set the Working directory to the repository root.
4. Click Run

To see the verbose trace, the bytecode needs a VERBOSE ON line in factorial.x.cod.

5 Assumptions

1. I assumed there is no division by 0 in test cases, so BOP division does not guard against it.
2. I assumed that if the program reaches the READ instructions, the user will type a valid integer; the prompt repeats until the input is an int.
3. For verbose display, I assumed every value can be displayed as a plain int, since the bytecode file does not record variable types.
4. I treated the work's verbose trace as the authoritative output format when single instructions seemed to spell spacing differently.

6 Implementation Discussion

The design decision that shaped everything else was making the Virtual Machine the only thing that touches the runtime stack and the heap. I chose this because letting each of the ~19 bytecodes operate on those structures directly would expose the fragile state in the program to every instruction class. By routing everything through the VM, each bytecode only makes requests, so the rules that keep memory consistent all live in one place and the instructions stay small. The trade-off is a layer of pass-through methods on the VM, which I accepted because the safety is more of a concern.

The bytecode names its jump targets by label. The simple version would be to scan the program for the matching LABEL every time a jump runs, but that repeats work on every jump and keeps the running machine tied to the source order. Instead Program.resolveAddress records each label's index once up front and stores the resolved instruction number on each jump. I did it this way so that at run time a jump is just setting the program counter.

I store all function values in one list and keep a second stack of frame pointers, where each pointer is the index where the current function's values start. I picked this because it lets every function call reuse the same list cheaply instead of allocating separate storage. The risk is that nothing physically stops an instruction from reaching into the function below it, so I enforce that limit in code, pop, store, and load all reject any offset that would cross the current frame's starting index.

I also followed a return-value convention rather than enforcing one. The agreement is that the caller sets up the arguments and the callee leaves its return value on top of the stack before returning. The machine can't enforce this, so the return logic match it exactly: it takes the top value, clears the rest of the current frame, removes the frame boundary, and pushes that one value back for the caller. I had to get this right for recursion to work, since only return value is meant to survive a return.

I kept the heap separate from the call stack and made it track freed blocks. Heap memory is its own structure because a heap block should still be usable after the function that created it returns. I store blocks in a map keyed by address and keep a separate set of freed addresses, so the heap can tell a never-allocated address apart from one that was freed, the assignment treats those as two different errors.

Finally, I let the machine own VERBOSE mode instead of the bytecodes. A bytecode isn't allowed to know whether verbose mode is on, so the VM decides after each instruction whether to print. The stack snapshot is built by the runtime stack, since it's the only part that knows where the frame boundaries are, but the VM decides when to print it. This keeps verbose logic out of the instruction classes.

6.1 UML Class Diagram



7 Project Reflection

This project encouraged me to implement encapsulation more than any project I have done till now. Once I started to write with bytecodes that only talk to the VM, it became easy to write it. It was code that made requests. The other thing I learned is how much of an interpreter is really about conventions, the calls and returns line up on the stack is an agreement that the instructions keep. Initially, I got a recursion bug, which was because of the broken agreement only.

Issues I ran into:

Recursion initially only returned values that were too small, because on return i leftover locals on the stack. I fixed it by keeping only the top values on return and clearing the rest of the leftover frame, boundaries, and pushing that one value back. Also i was not able to distinguish a freed block from a bad address, because freeing only removed the block, leaving no record of it. I resolved it by keeping a separate set of freed addresses so the heap reports use-after-free vs. invalid address correctly. Additionally, some little issues like verbose output did not match the trace, calls store lit return were printing on two lines, so I switched to tab so each prints on one line until they matched. Also, an additional stack line was printing after the verbose on, cause I printed the stack after every shown bytecode, including toggle, and resolved it by only printing the bytecode line for verbose instructions.

8 Project Conclusion and Results

The interpreter is complete and runs all the programs correctly:

1. fib.x.cod with input 7 produces 13.
2. factorial.x.cod with input 4 produces 24.
3. heapSumArray.x.cod produces 150.
4. heapPrintArray.x.cod correctly stores the integers the user types and prints them back in order.

All 19 bytecodes work, address resolution works, the runtime stack protects its frame boundaries, and the heap enforces required memory safety rules.

```
Project: interpreter-build4me2 main
Interpreter: factorial

Terminal: Local x
AI Agents:

interpreter-build4me2 main ? } printf "4\n" | java -cp build interpreter.Interpreter factorial.x.cod
VERBOSE ON
GOTO start<<1>>
[]
GOTO continue<<3>>
[]
ARGS 0
[] []
CALL Read Read()
[] []
Please enter an integer : READ
[] [4]
RETURN
[4]
ARGS 1
[] [4]
CALL factorial<<2>> factorial(4)
[] [4]
LOAD 0 n <load n>
[] [4,4]
LIT 2
[] [4,4,2]
BOP <
[] [4,0]
FALSEBRANCH else<<4>>
[] [4]
LOAD 0 n <load n>
[] [4,4]
LOAD 0 n <load n>
[] [4,4,4]
LIT 1
[] [4,4,4,1]
END
```

```
Project: interpreter-build4me2 main
Interpreter: factorial

Terminal: Local x
AI Agents:

interpreter-build4me2 main ? } mkdir -p build && find interpreter -name "*.java" > sources.txt && javac -d build @sources.txt && echo "BUILD OK"
BUILD OK

interpreter-build4me2 main ? } java -cp build interpreter.Interpreter heapSumArray.x.cod
printf "10\n20\n30\n" | java -cp build interpreter.Interpreter heapPrintArray.x.cod
150
Please enter an integer : Please enter an integer : Please enter an integer : 10
20
30

interpreter-build4me2 main ? } printf "7\n" | java -cp build interpreter.Interpreter fib.x.cod
Please enter an integer : 13

interpreter-build4me2 main ? }
```

```
Project: interpreter-build4me2
Terminal: Local x + v
factorial.x.cod x

[] [4,4] [3,3] [2,2] [1]
LIT 1
[] [4,4] [3,3] [2,2] [1,1]
RETURN factorial<<2>> EXIT factorial:1
[] [4,4] [3,3] [2,2,1]
BOP *
[] [4,4] [3,3] [2,2]
RETURN factorial<<2>> EXIT factorial:2
[] [4,4] [3,3,2]
BOP *
[] [4,4] [3,6]
RETURN factorial<<2>> EXIT factorial:6
[] [4,4,6]
BOP *
[] [4,24]
RETURN factorial<<2>> EXIT factorial:24
[24]
ARGS 1
[] [24]
CALL Write Write(24)
[] [24]
LOAD 0 dummyFormal <load dummyFormal>
[] [24,24]
24
WRITE
[] [24,24]
RETURN
[24]
POP 3
[]

interpreter-build4me2 main ? )
```

```
Project: interpreter-build4me2
Terminal: Local x + v
factorial.x.cod x

interpreter-build4me2 main ? ) printf "NEW 1\nFREE\nLIT 1\nLIT 0\nHLOAD\nHALT\n" > uaf_demo.cod && java -cp build interpreter.Interpreter uaf_demo.cod ; rm uaf_demo.cod
interpreter.virtualmachine.exceptions.HeapUseAfterFreeException: Heap block has already been freed: 1
interpreter.virtualmachine.exceptions.HeapUseAfterFreeException (Create breakpoint): Heap block has already been freed: 1
    at interpreter.virtualmachine.Heap.checkAccess(Heap.java:81)
    at interpreter.virtualmachine.Heap.load(Heap.java:54)
    at interpreter.virtualmachine.VirtualMachine.loadHeap(VirtualMachine.java:121)
    at interpreter.bytecodes.HeapLoadCode.execute(HeapLoadCode.java:20)
    at interpreter.virtualmachine.VirtualMachine.executeProgram(VirtualMachine.java:44)
    at interpreter.Interpreter.run(Interpreter.java:39)
    at interpreter.Interpreter.main(Interpreter.java:61)

interpreter-build4me2 main ? )
```